Troubleshooting Session 01

Account statements fail at peak time · WSO2 Integrator: MI 4.6.0

The problem report

The bank exposes account data through the `AccountStatementService` data service. The operations team reports:

- On a normal day a statement takes about 3 seconds. Nobody complains.
- At month end, when many customers download statements at the same time, statements take up to 30 seconds and many fail with **HTTP 500**.
- During that time even the simple account lookup, which normally answers at once, becomes very slow.
- The database team says MySQL is healthy and the statement query itself has not changed.

Your job: reproduce the problem, collect evidence, find the cause, fix it and prove the fix with the same load.

You need: MI 4.6.0 started with JDK 21, MySQL 8 or later, `Shared Resources/MySQL JDBC Driver/mysql-connector-j-8.2.0.jar`. **Files** (this folder): `setup-bank-db.sql`, `AccountStatementService.dbs`, `LoadClient.java`.

Commands: each step shows macOS / Linux (Terminal) and Windows (PowerShell). On Windows, use `curl.exe` (plain `curl` means something else in PowerShell), and make sure `mysql` is on the PATH (for example `C:\Program Files\MySQL\MySQL Server 8.0\bin`).

Part A — Set up

1 Create the bank database (as the MySQL root user). It creates `bankdb` with 5 accounts and 13 transactions, and the user `bankuser`. Open a terminal in this folder first.

macOS / Linux

```
mysql -u root -p < setup-bank-db.sql
```

Windows (PowerShell has no <, so let cmd run it)

```
cmd /c "mysql -u root -p < setup-bank-db.sql"
```

2 Copy `mysql-connector-j-8.2.0.jar` to `<MI_HOME>/lib` (Windows: `<MI_HOME>\lib`).

3 Create the folder `<MI_HOME>/repository/deployment/server/dataservices` (a fresh pack does not have it) and copy `AccountStatementService.dbs` into it.

CC BY 4.0

4 Start MI and wait for `WSO2 Micro Integrator started`:

macOS / Linux

```
cd <MI_HOME>/bin
./micro-integrator.sh
```

Windows

```
cd <MI_HOME>\bin
.\micro-integrator.bat
```

Part B — Normal day

How to run LoadClient.java

`LoadClient.java` is a small Java program that sends many requests at the same time and prints every response and a summary. It needs **JDK 11 or later** and **no build step**: `java` compiles and runs the single file in one go. The JDK 21 that runs MI is enough.

Arguments: `java LoadClient.java <url> <users> <requestsPerUser>`. Without arguments: the statement of account 1001, 80 users at the same time, 1 request each.

1. Open a terminal **in this folder** (the folder with `LoadClient.java`) and check Java:

macOS / Linux

```
cd "/path/to/Troubleshooting Session 01"
java -version
```

Windows

```
cd "C:\path\to\Troubleshooting Session 01"
java -version
```

2. If `java` is not found or is older than 11, point this terminal to your JDK 21:

macOS

```
export JAVA_HOME=$(/usr/libexec/java_home -v 21)
export PATH="$JAVA_HOME/bin:$PATH"
```

Linux

```
export JAVA_HOME=/path/to/jdk-21
export PATH="$JAVA_HOME/bin:$PATH"
```

Windows

```
$env:JAVA_HOME = "C:\path\to\jdk-21"
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
```

3. Set the service address once in each terminal; the steps below use it:

macOS / Linux

```
BASE=http://localhost:8290/services/AccountStatementService
```

CC BY 4.0

Windows

```
$BASE = "http://localhost:8290/services/AccountStatementService"
```

5 Call both resources once and note the time of each:

macOS / Linux

```
curl -s -w "\n%{time_total}s\n" -H "Accept: application/json" \
  $BASE/account/1001
curl -s -w "\n%{time_total}s\n" -H "Accept: application/json" \
  $BASE/statement/1001
```

Windows

```
curl.exe -s -w "\n%{time_total}s\n" -H "Accept: application/json" `
  "$BASE/account/1001"
curl.exe -s -w "\n%{time_total}s\n" -H "Accept: application/json" `
  "$BASE/statement/1001"
```

6 Run a small load: 5 users at the same time, one request each:

macOS / Linux

```
java LoadClient.java $BASE/statement/1001 5 1
```

Windows

```
java LoadClient.java "$BASE/statement/1001" 5 1
```

Each line shows the user, the HTTP status and the time; the summary shows successful, failed and min / avg / max time.

Part C — Month end

7 Terminal 1: run the default load (80 users at the same time):

macOS / Linux

```
java LoadClient.java $BASE/statement/1001
```

Windows

```
java LoadClient.java "$BASE/statement/1001"
```

CC BY 4.0

8 Terminal 2, while the load is running: set `BASE` again (step 3 of the box above), call the account lookup and note the time:

macOS / Linux

```
curl -s -w "\n%{time_total}s\n" -H "Accept: application/json" \
  $BASE/account/1002
```

Windows

```
curl.exe -s -w "\n%{time_total}s\n" -H "Accept: application/json" `
  "$BASE/account/1002"
```

9 Write down: the summary (successful, failed, min / avg / max time), the HTTP status of the failures, and the time of the account lookup. Compare with Part B.

Part D — Collect evidence

Run the load from Part C again for each check below. You have about 30 seconds while it runs.

10 **Server log.** Open `<MI_HOME>/repository/logs/wso2carbon.log` (or `wso2error.log`). Find the first `ERROR` and read every `Nested Exception` down to the last one. Note the exception class and the numbers in square brackets.

11 **Thread dump.** While the load runs, take a thread dump of MI and look at what the worker threads (`PassThroughMessageProcessor-*`) are doing. If more than one MI runs, `jps -lv` shows the folder of each.

macOS / Linux

```
jps -l | grep -i bootstrap                      # MI process id
jstack <pid> > td1.txt
grep -c "PassThroughMessageProcessor" td1.txt   # worker threads
grep -A14 "PassThroughMessageProcessor" td1.txt | grep "at org" \
  | sort | uniq -c | sort -rn | head            # most common frames
```

Windows

```
jps -l | findstr /i bootstrap                   # MI process id
jstack <pid> > td1.txt
(Select-String td1.txt -Pattern "PassThroughMessageProcessor").Count
Select-String td1.txt -Pattern "PassThroughMessageProcessor" -Context 0,14 |
  ForEach-Object { $_.Context.PostContext } |
  Where-Object { $_ -match "at org" } |
  Group-Object | Sort-Object Count -Descending |
  Select-Object -First 10 Count, Name
```

Count how many workers are running SQL in MySQL, and how many are waiting. In which class and method do the waiting ones wait?

CC BY 4.0

12 **Database side.** While the load runs, look at the connections from MI (the same commands work on macOS, Linux and Windows):

```
mysql -u root -p -e "SELECT USER, COMMAND, TIME, STATE
  FROM information_schema.PROCESSLIST WHERE USER='bankuser'"
mysql -u root -p -e "SHOW VARIABLES LIKE 'max_connections'"
```

How many connections does MI have open to `bankdb`? What are they doing? Is MySQL anywhere near its limit?

13 **Configuration.** Open `AccountStatementService.dbs` and read the `<config>` section. Which property matches a number you saw in the log, the thread dump and the MySQL output?

14 Write down your conclusion in one sentence: what limits the service, and why the account lookup slows down too.

Part E — Fix and prove

15 Change the data service to remove the limit you found. Edit the `.dbs` file in `<MI_HOME>/repository/deployment/server/dataservices` and save it. MI redeploys a changed data service by itself (the log shows `Removing Axis2 Service: AccountStatementService`). Wait about 20 seconds and call the statement once, as in step 5, before you run the load.

16 Run the same load as in Part C, and the account lookup in terminal 2. Write down the summary and the lookup time, and compare with Part C.

17 Check the database side again (step 12): how many connections does MI keep open after the load has finished?

Questions for discussion

- Which evidence pointed to the cause first: the HTTP response, the log, the thread dump or the database?
- Why did the account lookup slow down, although its query is fast?
- How did you choose the new value? What sets the upper limit for it (think about MySQL, and about two or more MI nodes using the same database)?
- Raising the setting fixed the symptom. What other changes would also help, on the MI side and on the database side?
- What would you monitor in production to see this coming before customers do?

Clean up

18 Stop MI (Ctrl+C). Optionally drop the database (macOS, Linux and Windows):

```
mysql -u root -p -e "DROP DATABASE bankdb; DROP USER 'bankuser'@'localhost'"
```

CC BY 4.0